PREDICTING HEART DISEASES USING MACHINE LEARNING AND DIFFERENT DATA CLASSIFICATION TECHNIQUES

CardioSense: a web application comparing ANOVA, Chi-square and Mutual Information feature selection

A project report submitted in partial fulfilment of the requirements for the degree of

[Bachelor of Technology] in [Computer Science and Engineering]

by

[Student name 1] ([Register number])

[Student name 2] ([Register number])

[Student name 3] ([Register number])

Under the guidance of

[Guide name], [Designation]

[Department], [College / University name]

[Month Year]

Abstract

Heart disease is the leading cause of death worldwide, and many of the warning signs are already present in routine clinical measurements. This project builds and evaluates a system, **CardioSense**, that predicts the presence of heart disease from 13 standard attributes of the UCI Cleveland heart disease dataset (303 patients), and compares three filter-based feature-selection techniques: the ANOVA F-test, the Chi-square test and Mutual Information.

A data audit found that the widely redistributed copy of the dataset had every class label reversed; all 303 rows were matched against the original UCI file and the labels were corrected. A reproducible pipeline then holds out 61 patients for testing before any learning step, balances the training set with a tuned SMOTEENN (SMOTENC oversampling followed by Edited Nearest Neighbours cleaning), selects 8, 11 and 10 features with the three methods, trains eleven classifiers (including voting and stacking ensembles), and keeps the best per method by 5-fold cross-validation.

The best models reach a cross-validated accuracy of 85.5% (ANOVA with KNN), 84.6% (Chi-square with Naive Bayes) and 83.8% (Mutual Information with Naive Bayes), with test-set ROC AUC between 0.84 and 0.87. The three methods are within normal fold-to-fold variation of each other, and three additional stacking ensembles did not improve on them. The project also shows that a 100% accuracy reported by earlier notebooks came from resampling before the train/test split (data leakage).

The models are deployed in a Flask web application with secure sign-up (hashed passwords, email OTP, sessions), single-patient prediction with an explanation of each value, side-by-side comparison of the three methods on the same patient, dataset analytics, live model-evaluation charts and per-user history. An automated test suite and a 45-case responsive layout test pass in full.

Keywords: heart disease prediction, feature selection, ANOVA, Chi-square, mutual information, SMOTEENN, stacking, cross-validation, data leakage, Flask.

TABLE OF CONTENTS

Abstract ..... **i**

1. INTRODUCTION **1**

1.1 Background 1

1.2 Motivations 1

1.3 Scope of the Project 2

2. PROJECT DESCRIPTION AND GOALS **3**

2.1 Literature Review 3

2.2 Research Gap 4

2.3 Objectives 5

2.4 Problem Statement 5

2.5 Project Plan 5

3. TECHNICAL SPECIFICATION **6**

3.1 Requirements 6

3.1.1 Functional 6

3.1.2 Non-Functional 6

3.2 Feasibility Study 7

3.2.1 Technical Feasibility 7

3.2.2 Economic Feasibility 7

3.2.3 Social Feasibility 7

3.3 System Specification 8

3.3.1 Hardware Specification 8

3.3.2 Software Specification 8

4. DESIGN APPROACH AND DETAILS **9**

4.1 System Architecture 9

4.2 Design 9

4.2.1 Data Flow Diagram 9

4.2.2 Use Case Diagram 10

4.2.3 Class Diagram 11

4.2.4 Sequence Diagram 12

5. METHODOLOGY AND TESTING **13**

5.1 Module Description 13

5.1.1 Data Preparation and Label Verification 13
5.1.2 Train/Test Split 13
5.1.3 Resampling with SMOTEENN 13
5.1.4 Feature Selection 13
5.1.5 Model Training and Selection 14
5.1.6 Web Application Modules 14
5.2 Testing 15
6. PROJECT DEMONSTRATION 17
7. RESULT AND DISCUSSION 23
7.1 Classifier Results per Method 23
7.2 Comparison of the Three Methods 24
7.3 Effect of SMOTEENN 25
7.4 Stacking Experiment 25
7.5 Comparison with the Original Notebooks 25
7.6 Cost Analysis 26
8. CONCLUSION 27
9. REFERENCES 28
APPENDIX A – SAMPLE CODE 30
A.1 Tuned SMOTEENN (train.py) 30
A.2 Cross-validated training and model choice (train.py) 30
A.3 Input validation (app.py) 30
A.4 Prediction route (app.py) 31

1. INTRODUCTION

1.1 Background

Cardiovascular diseases (CVDs) are the leading cause of death globally; the World Health Organization estimates that about 17.9 million people die from CVDs each year [1]. Coronary artery disease, the most common form, develops over years and is often detected late. Diagnosis normally relies on a combination of symptoms, risk factors and tests such as the electrocardiogram (ECG), exercise stress testing, thallium scanning and coronary angiography.

Many of the measurements collected during this work-up are numeric or categorical values recorded in patient files: age, resting blood pressure, serum cholesterol, maximum heart rate during exercise, ST-segment depression, and so on. Machine learning (ML) can learn the relationship between these values and a confirmed diagnosis, and so act as a low-cost first screen that flags patients who need further investigation.

The best-known public benchmark for this task is the Cleveland heart disease database collected by Detrano et al. [2] and distributed by the UCI Machine Learning Repository [3]. It contains 303 patients described by 13 clinical attributes and a diagnosis of angiographic heart disease. Its small size and mix of numeric and categorical attributes make it a good test bed for comparing preprocessing and classification techniques.

1.2 Motivations

- **Fewer, better measurements.** If a small subset of the 13 attributes predicts as well as all of them, screening becomes cheaper and simpler. Feature selection identifies that subset, and different selection techniques can disagree; comparing them is the core question of this project.
- **Trustworthy numbers.** Many published and student works on this dataset report accuracies above 95%, sometimes 100%. Such figures often come from evaluation mistakes, especially resampling or selecting features before splitting off the test set (data leakage) [4]. We wanted results that hold up when the experiment is re-run.
- **Understandable results.** A bare "positive/negative" output is hard to trust. A tool that shows the risk, compares methods on the same patient, and explains which values point toward disease is more useful for learning and discussion.

- **Secure, usable software.** The original codebase stored passwords as plain text and embedded an email password in the source. A health-related application should handle personal data responsibly.

1.3 Scope of the Project

In scope:

- Binary prediction (heart disease present or absent) from the 13 UCI Cleveland attributes.
- Three filter feature-selection methods (ANOVA F-test, Chi-square, Mutual Information) and eleven classifiers per method.
- Class balancing with SMOTEENN applied to training data only; evaluation by a held-out test set and 5-fold cross-validation.
- A web application for registration, prediction, comparison, dataset analytics, model insights and history, usable on desktop and mobile.

Out of scope: clinical deployment or diagnosis, other datasets or hospitals, imaging or ECG signal analysis, and real-time monitoring. The system is a study and screening aid, not a medical device.

2. PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

Dataset and clinical background. Detrano et al. [2] collected the Cleveland, Hungarian, Swiss and Long Beach VA databases and evaluated a probability algorithm for diagnosing coronary artery disease across the four sites, establishing the attribute set used by almost all later work. Only the Cleveland subset is commonly used for ML because the other sites have many missing values in key attributes such as the number of major vessels (ca) and the thallium scan result (thal) [3].

Feature selection. Guyon and Elisseeff [5] describe the three families of feature selection: filter methods that score each feature independently of the classifier, wrapper methods that search subsets using a model, and embedded methods that select during training. Filter methods are fast and classifier-independent, which suits a comparison study. The ANOVA F-test compares feature means across classes; the Chi-square test measures dependence between a non-negative feature and the class; Mutual Information, estimated with nearest-neighbour methods [6], captures non-linear dependence. Noroozi et al. [7] compared filter, wrapper and embedded selection on heart disease data and found that the benefit of feature selection depends strongly on the classifier, which motivates testing several classifiers per method.

Class imbalance and resampling. Chawla et al. [8] introduced SMOTE, which creates synthetic minority-class samples by interpolating between neighbours. Wilson [9] proposed Edited Nearest Neighbours (ENN), which removes samples whose neighbours mostly belong to another class. Batista et al. [10] showed that combining them (SMOTE+ENN) can improve results by both balancing and cleaning the data. The imbalanced-learn library [11] implements these methods in a pipeline that can be cross-validated correctly.

Classifiers and ensembles. Random forests [12], AdaBoost [13], gradient boosting implementations XGBoost [14] and LightGBM [15], and stacked generalisation [16], which trains a meta-model on the predictions of several base models, are standard choices for tabular medical data. Simpler models such as Naive Bayes, logistic regression and k-nearest neighbours often perform competitively on small datasets. All are available in scikit-learn [17].

Evaluation pitfalls. Kaufman et al. [4] define leakage as information from outside the training data entering model building, and show that it produces optimistic results that do not hold in

practice. Oversampling or selecting features before the train/test split is a common form of leakage in imbalanced-data studies.

Table 2.1: Summary of the literature reviewed

Ref.	Work	Key idea	Use in this project
[2]	Detrano et al., 1989	Multi-centre heart disease data and diagnostic algorithm	Source of the dataset and attributes
[5]	Guyon & Elisseeff, 2003	Filter, wrapper and embedded feature selection	Choice of filter methods for a fair comparison
[6]	Kraskov et al., 2004	Nearest-neighbour estimate of mutual information	Basis of the MI scores
[7]	Noroozi et al., 2023	Feature selection effects vary by classifier	Eleven classifiers per method
[8]–[10]	Chawla 2002; Wilson 1972; Batista 2004	SMOTE, ENN, and their combination	Tuned SMOTEENN on training data
[12]–[16]	Breiman; Freund & Schapire; Chen; Ke; Wolpert	Tree ensembles, boosting, stacking	Classifier and ensemble choices
[4]	Kaufman et al., 2012	Leakage gives optimistic results	Split first; resample inside CV folds

2.2 Research Gap

- **Leaky evaluation.** Many works on this dataset oversample the whole dataset before splitting, so synthetic copies of test patients appear in training and accuracy is inflated. The original notebooks of this project reported 100% for the stacking model for this reason.
- **Unverified data.** Redistributed copies of the dataset are used without checking them against the original source. We found that one popular copy has every label reversed, which does not change accuracy but inverts every prediction.
- **Point estimates without uncertainty.** Results are usually a single test-split accuracy. On about 60 test patients, one patient moves accuracy by 1.6 points, so differences between methods are often within noise.
- **No side-by-side view.** Tools rarely let a user see how different feature-selection methods treat the same patient, or how a prediction compares with the real diagnosis.

2.3 Objectives

1. Audit the dataset against its original source and correct any errors.
2. Build a reproducible training pipeline that keeps the test set completely separate from every learning step.
3. Compare ANOVA, Chi-square and Mutual Information feature selection using eleven classifiers each, and report cross-validated accuracy with its spread.
4. Measure whether SMOTEENN resampling and stronger stacking ensembles improve the results.
5. Deploy the best model per method in a secure, responsive web application with prediction, comparison, explanation and analytics.
6. Verify the system with automated and responsive-layout tests.

2.4 Problem Statement

> Given 13 routine clinical measurements of a patient, predict whether the patient has heart disease, and determine which of three feature-selection methods (ANOVA, Chi-square, Mutual Information) yields the most accurate classifier, using an evaluation that is free of data leakage, and make the result usable through a secure web application.

2.5 Project Plan

The work was organised in the phases below. [Adjust the weeks to your actual schedule.]

Table 2.2: Project plan

Phase	Weeks	Activities	Deliverable
1. Study	1–2	Literature review, dataset study, problem definition	Synopsis
2. Audit	3	Security review of the code base; dataset check against UCI	Security report; corrected dataset
3. Pipeline	4–6	Split, SMOTEENN, feature selection, 11 classifiers, cross-validation	train.py, saved models, metrics.json
4. Application	7–9	Flask app: authentication, prediction, comparison, analytics, history; UI design	Working web application
5. Testing	10	Automated suite, responsive layout tests, manual tests	Test report
6. Documentation	11–12	Report, presentation, demonstration	Final report and slides

3. TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

Table 3.1: Functional requirements

ID	Requirement
FR1	A visitor can register with username, name, email, phone and password, confirmed by a 6-digit code sent by email.
FR2	A registered user can log in and out; all prediction and analysis pages require login.
FR3	The user can choose a feature-selection method (ANOVA, Chi-square, MI) and enter the patient values that method uses.
FR4	The system validates every value (range for numbers, allowed codes for categories) and rejects invalid input with a message.
FR5	The system returns a prediction (positive/negative) with an estimated risk percentage.
FR6	The user can fill the form with a random real patient from the dataset; the result then shows the real diagnosis and whether the model was correct.
FR7	The user can run the same patient through all three methods and compare the results side by side.
FR8	For each result, the system explains which values point toward or away from heart disease, based on the dataset.
FR9	The system shows dataset analytics: class balance, averages by diagnosis, disease share per group, SMOTEENN stages and a feature glossary.
FR10	The system shows model insights: feature-selection scores, an 11-classifier leaderboard, ROC curve, confusion matrix and resampling comparison.
FR11	Each prediction is saved to the user's history; the user can view and clear it.
FR12	Results can be printed or saved as PDF; the interface offers light and dark themes.

3.1.2 Non-Functional

Table 3.2: Non-functional requirements

Quality	Requirement
Security	Passwords hashed (scrypt via Werkzeug); secrets only in environment variables; signed session cookies; OTP expires in 5 minutes and allows 5 attempts; credentials sent by POST only.
Performance	A prediction returns in well under a second; models are loaded once at start-up; retraining takes about 3 minutes on a laptop.
Usability	Responsive layout from 375 px phones to desktops; keyboard-operable dropdowns; info notes on every input; light and dark themes.

Quality	Requirement
Reliability	Server-side validation of every input; invalid or tampered requests are rejected without errors.
Maintainability	One source of truth (metrics.json) drives forms and charts; automated test suite; plain HTML/CSS/JS front end.
Reproducibility	Fixed random seed (42); pinned package versions in requirements files; training and experiments are scripts.
Portability	Python 3.10 or newer on Windows, macOS or Linux; any modern browser.

3.2 Feasibility Study

3.2.1 Technical Feasibility

All components are mature, open-source and run on an ordinary laptop: Python, scikit-learn, imbalanced-learn, LightGBM and XGBoost for modelling, Flask for the web server and SQLite for storage. The dataset is small (302 unique patients), so training all 11 classifiers for 3 methods with cross-validation completes in about three minutes on the development machine, and a prediction takes milliseconds. No GPU or cloud service is required.

3.2.2 Economic Feasibility

The project has no licensing cost: every library is free, the dataset is public, and the application runs on an existing computer. The only external service, Gmail SMTP for OTP emails, is free within normal sending limits. A public deployment would add only a small hosting cost (see the cost analysis in Chapter 7).

3.2.3 Social Feasibility

A screening aid that uses routine measurements can raise awareness of heart-disease risk factors and support teaching. The application states clearly on every result that it is not a diagnosis. Personal data is limited to account details, stored locally with hashed passwords, and each user sees only their own history.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.3: Hardware specification

Component	Minimum	Development machine (tested)
Processor	Any 64-bit dual-core CPU	12th Gen Intel Core i5-1240P (12 cores)
Memory	4 GB RAM	16 GB RAM
Storage	2 GB free (Python environment and project)	SSD
Display	375 px wide (phone) or larger	Laptop display
Network	Internet for installing packages and sending OTP emails	Wi-Fi

3.3.2 Software Specification

Table 3.4: Software specification

Software	Version	Purpose
Operating system	Windows 11 (also macOS / Linux)	Development and running
Python	3.13 (3.10 or newer)	Programming language
Anaconda / conda	Any recent version	Environment management (optional)
Flask / Werkzeug	3.1.3 / 3.1.9	Web framework, password hashing
scikit-learn	1.9.1	Feature selection, classifiers, metrics
imbalanced-learn	0.14.2	SMOTENC, ENN, resampling pipelines
LightGBM / XGBoost	4.7.0 / 3.4.1	Gradient boosting (stacking meta-model; XGBoost classifier)
NumPy / SciPy / joblib	2.5.4 / 1.18.1 / 1.6.0	Numerics and model files
SQLite	Built into Python	Users and prediction history
Browser	Chrome, Edge, Firefox or Safari	User interface

4. DESIGN APPROACH AND DETAILS

4.1 System Architecture

CardioSense follows a three-layer architecture with a separate offline training stage (Figure 4.1). The **presentation layer** consists of Jinja2 templates rendered in the browser, one stylesheet (light and dark themes) and one small JavaScript file. The **application layer** is a Flask application that handles authentication, input validation, prediction, comparison, analytics and history. The **data and model layer** holds the SQLite database, the saved models and the metrics file.

Training is offline: `train.py` reads the dataset, runs the full pipeline and writes the three model files plus `metrics.json`. The web application reads its feature lists, leaderboards and charts from that file, so retraining updates the whole interface without code changes.

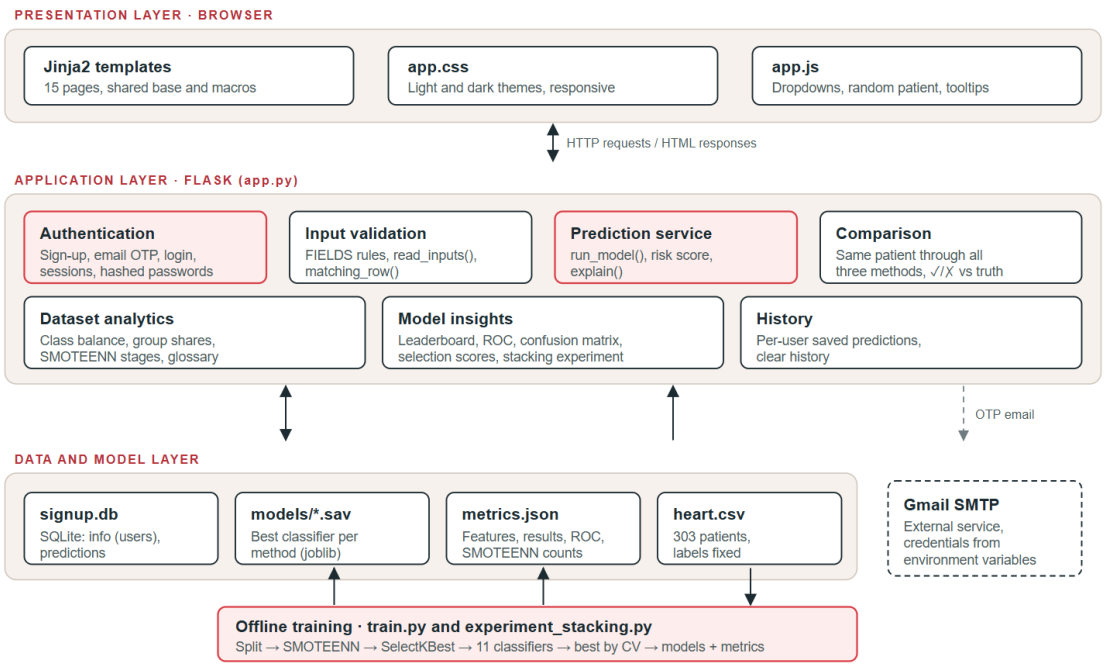


Figure 4.1: System architecture of CardioSense

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.2 shows the context (level 0) diagram: the user exchanges registration details, credentials and patient values with the system, and the system asks the email service to deliver

the one-time password. Figure 4.3 decomposes the system into six processes and four data stores.

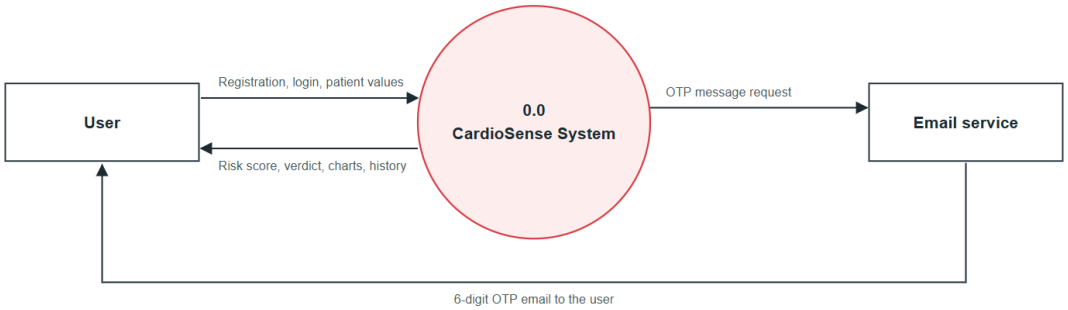


Figure 4.2: Data flow diagram, level 0 (context diagram)

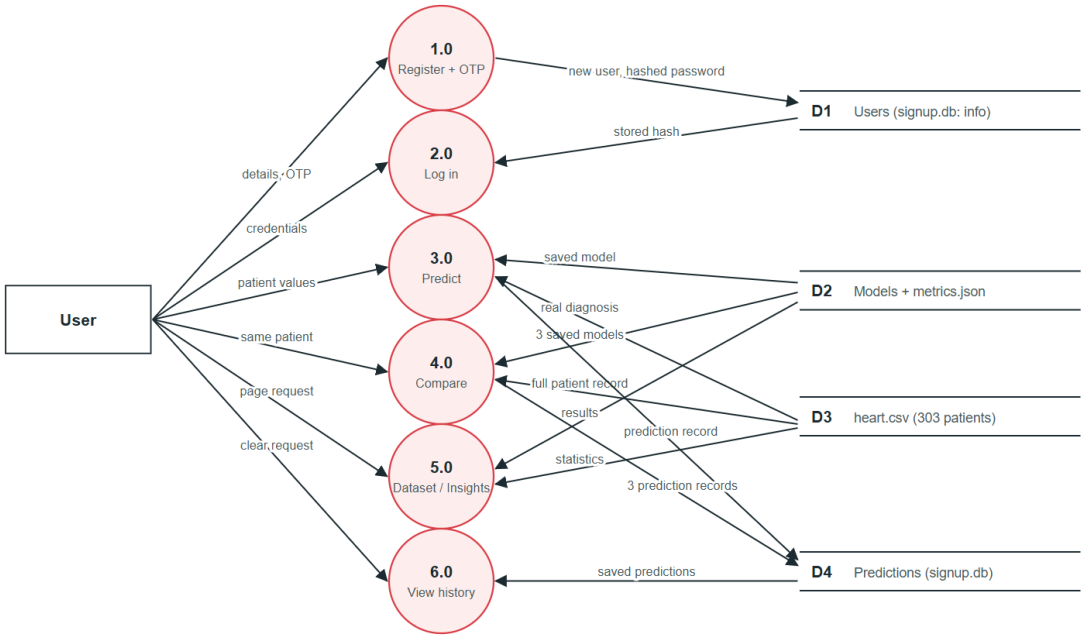


Figure 4.3: Data flow diagram, level 1

4.2.2 Use Case Diagram

The main actor is the user, who registers, logs in, predicts, compares, explores analytics and manages history. The email service is a secondary actor in registration. The developer retrains the models, runs the stacking experiment and runs the tests (Figure 4.4).

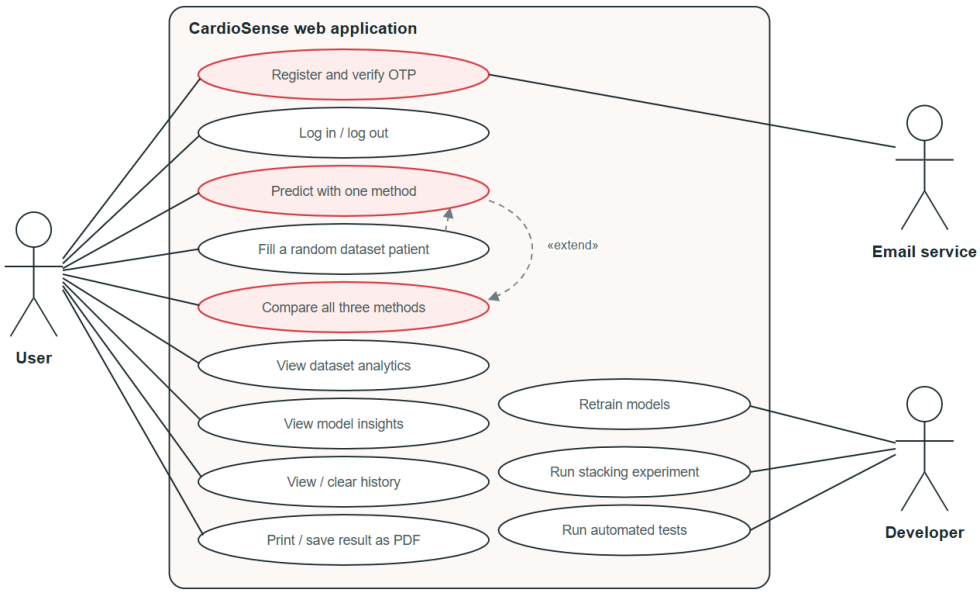


Figure 4.4: Use case diagram

4.2.3 Class Diagram

The application is written in a functional style in Python, so Figure 4.5 shows its modules and data structures as classes: the Flask application, the per-method model registry (MODELS), the input rules (FIELDS), the database tables for users and predictions, the dataset statistics and the training module.

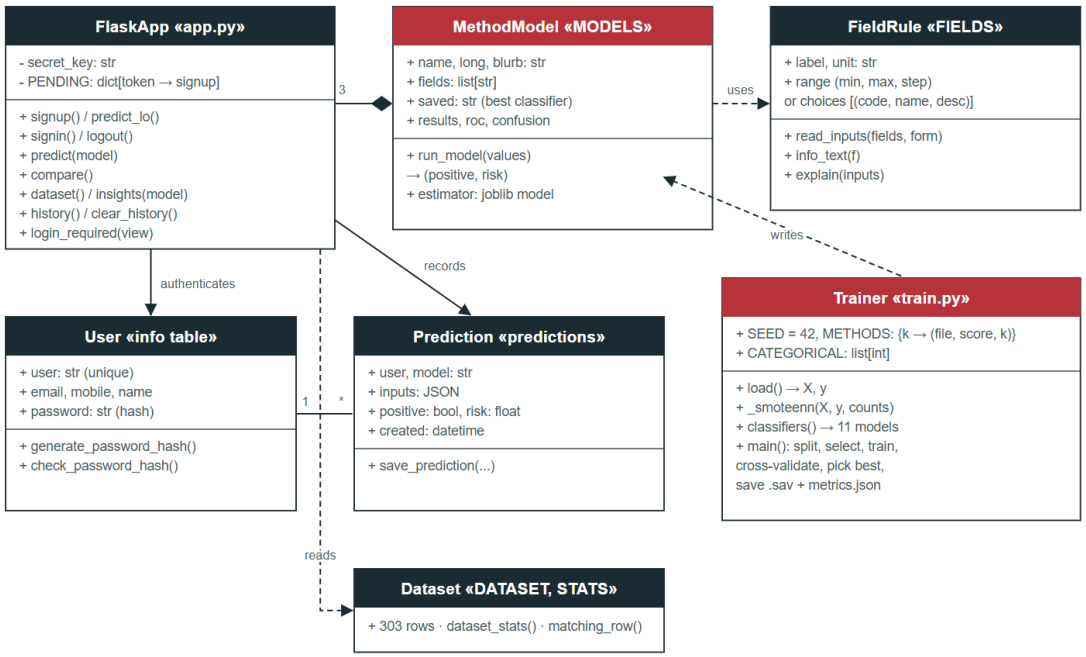


Figure 4.5: Class diagram (modules and data structures)

4.2.4 Sequence Diagram

Figure 4.6 shows the main interaction: a user fills a random dataset patient, predicts with one method, then compares the same patient across all three methods.

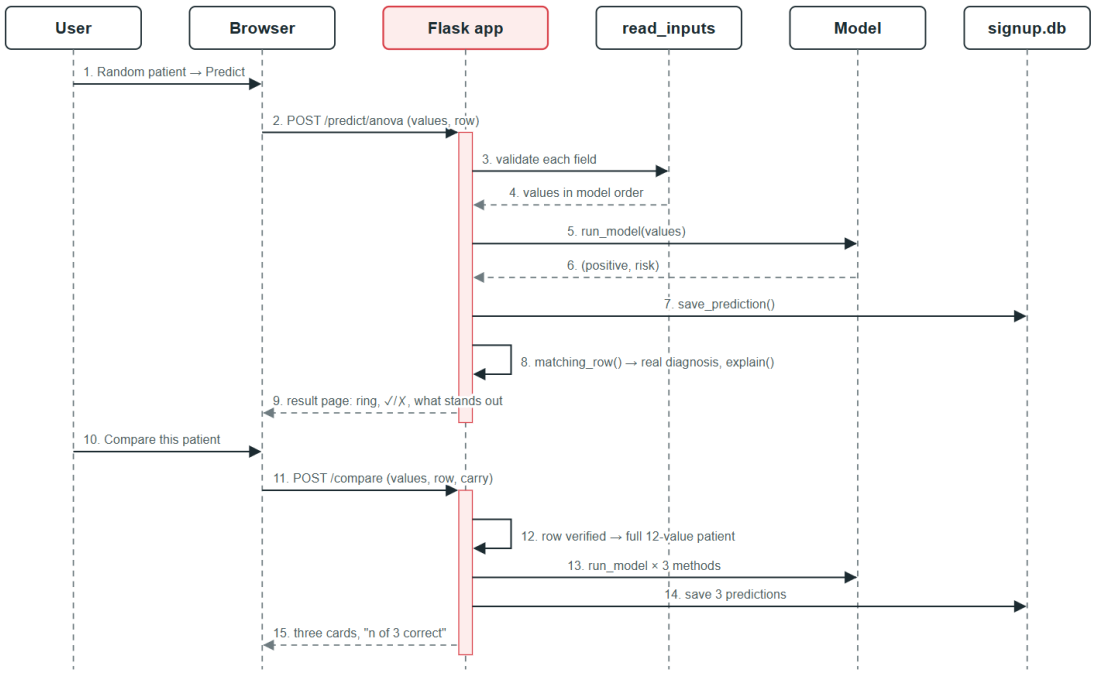


Figure 4.6: Sequence diagram for prediction and comparison

5. METHODOLOGY AND TESTING

5.1 Module Description

5.1.1 Data Preparation and Label Verification

The dataset (`archive/heart.csv`) has 303 rows and 14 columns. One exact duplicate is dropped, leaving 302 patients. Every row was matched against the original UCI file `processed.cleveland.data` on eight attributes stored identically in both (age, sex, blood pressure, cholesterol, fasting sugar, maximum heart rate, angina, ST depression). All 303 rows matched and all 303 labels were reversed: the "disease" group had a higher maximum heart rate (158.5 vs 139.1 bpm) and far less exercise angina (14% vs 55%), the opposite of clinical expectation. The labels were flipped so that 1 means disease; the original file is kept. The same matching decoded the categorical codes into their clinical meanings (e.g. chest pain type 0 = asymptomatic).

5.1.2 Train/Test Split

The data is split once, stratified by class, into 241 training and 61 test patients (80/20, seed 42). The test patients are never used for resampling, feature selection or model choice.

5.1.3 Resampling with SMOTEENN

The default SMOTEENN removed about three quarters of the training set because unscaled cholesterol dominated its distance calculations. The tuned version scales features first, uses SMOTENC so categorical attributes keep valid codes, and uses ENN with `kind_sel="mode"`. On the training set, SMOTE adds 21 synthetic patients with disease (110 → 131) and ENN removes 21 (20 real, 1 synthetic), leaving 131 healthy and 110 diseased patients, 20 of them synthetic.

5.1.4 Feature Selection

`SelectKBest` is fitted on the resampled training data with three scoring functions: `f_classif` (ANOVA, k = 8), `chi2` (k = 11) and `mutual_info_classif` (k = 10, fixed random state). Table 5.1 lists the features each method keeps. All three agree on eight core signs; none selects fasting blood sugar.

Table 5.1: Features selected by each method

Feature	ANOVA	Chi-square	MI
Age	–	Yes	–
Sex	Yes	Yes	Yes
Chest pain type	Yes	Yes	Yes
Resting blood pressure	–	Yes	Yes
Serum cholesterol	–	Yes	Yes
Fasting blood sugar	–	–	–
Resting ECG	–	–	–
Max heart rate	Yes	Yes	Yes
Exercise angina	Yes	Yes	Yes
ST depression	Yes	Yes	Yes
ST slope	Yes	Yes	Yes
Major vessels	Yes	Yes	Yes
Thallium scan	Yes	Yes	Yes

5.1.5 Model Training and Selection

Eleven classifiers are trained per method: Naive Bayes, SVM, XGBoost, AdaBoost, Bagging, Decision Tree, KNN (k = 3), Random Forest, Logistic Regression, soft Voting of the first nine, and Stacking (boosted decision stumps and extra trees combined by LightGBM). SVM, KNN and logistic regression are preceded by a standard scaler. Each classifier is cross-validated with 5 stratified folds, with resampling and feature selection repeated inside every fold through an imbalanced-learn pipeline. The classifier with the best cross-validated accuracy is saved for the method.

5.1.6 Web Application Modules

Table 5.2: Application modules

Module	Responsibility	Main functions / pages
Authentication	Sign-up with email OTP, login, logout, session protection	signup, predict_lo, signin, login_required
Input validation	Range and code checks for all 12 inputs	FIELDS, read_inputs, matching_row
Prediction	Risk and verdict from the saved model; dataset explanation	run_model, explain, /predict/<method>

Module	Responsibility	Main functions / pages
Comparison	Same patient through all three methods, correct/wrong vs truth	/compare
Analytics	Dataset statistics and live model-evaluation charts	/dataset, /insights/<method>
History	Per-user saved predictions, clear history	save_prediction, /history
Interface	Custom dropdowns, toggles, info notes, themes, print	templates, app.css, app.js

5.2 Testing

Testing combined an automated suite (`test_app.py`, run with the Flask test client against a temporary database), a responsive layout test that loads every page at three screen widths, and manual tests in the browser.

Table 5.3: Test cases

ID	Test case	Expected result	Result
T1	Start-up with plain-text passwords in the database	Passwords are hashed; old users can still log in	Pass
T2	Open a protected page without logging in	Redirect to login	Pass
T3	Send credentials in the URL (GET /signin)	Rejected (HTTP 405)	Pass
T4	Register; check the emailed code	6-digit numeric OTP	Pass
T5	Enter a wrong OTP five times	Sign-up cancelled	Pass
T6	Second visitor signs up while first is verifying	First visitor's account is created correctly	Pass
T7	Register an existing username	Rejected with a message	Pass
T8	Validate all 303 dataset rows for each method	Every row accepted	Pass
T9	Text in a number field; value out of range	Rejected with a message, no crash	Pass
T10	Predict with each method	Page shows POSITIVE or NEGATIVE with risk	Pass
T11	Unknown method in the URL	HTTP 404	Pass
T12	Saved models vs metrics.json	Feature counts match; saved model is the best by CV	Pass
T13	Dataset patient → result → compare	Real diagnosis shown; all 3 models see the full record	Pass

ID	Test case	Expected result	Result
T14	Edit a value after picking a random patient	No longer linked to the dataset row	Pass
T15	Typed patient → compare	Compare form opens pre-filled	Pass
T16	Explanation for a sick patient	Age, angina, vessels, heart rate flagged as risk	Pass
T17	Clear history	Only the user's own entries are removed	Pass
T18	Insights page	ROC, SMOTEENN and stacking sections render	Pass

Responsive layout test. Fifteen pages, including a submitted result and a comparison, were loaded at 1280 px, 820 px and 375 px. For each, the test checked that the page does not scroll sideways, no element overflows its card, every dropdown is built and no image is broken: **45 of 45 passed**, with no browser console errors. Dark mode, keyboard operation of the dropdowns and the info notes were checked manually.

6. PROJECT DEMONSTRATION

This chapter walks through the application as a user sees it. The screenshots were taken from the running system at desktop size.

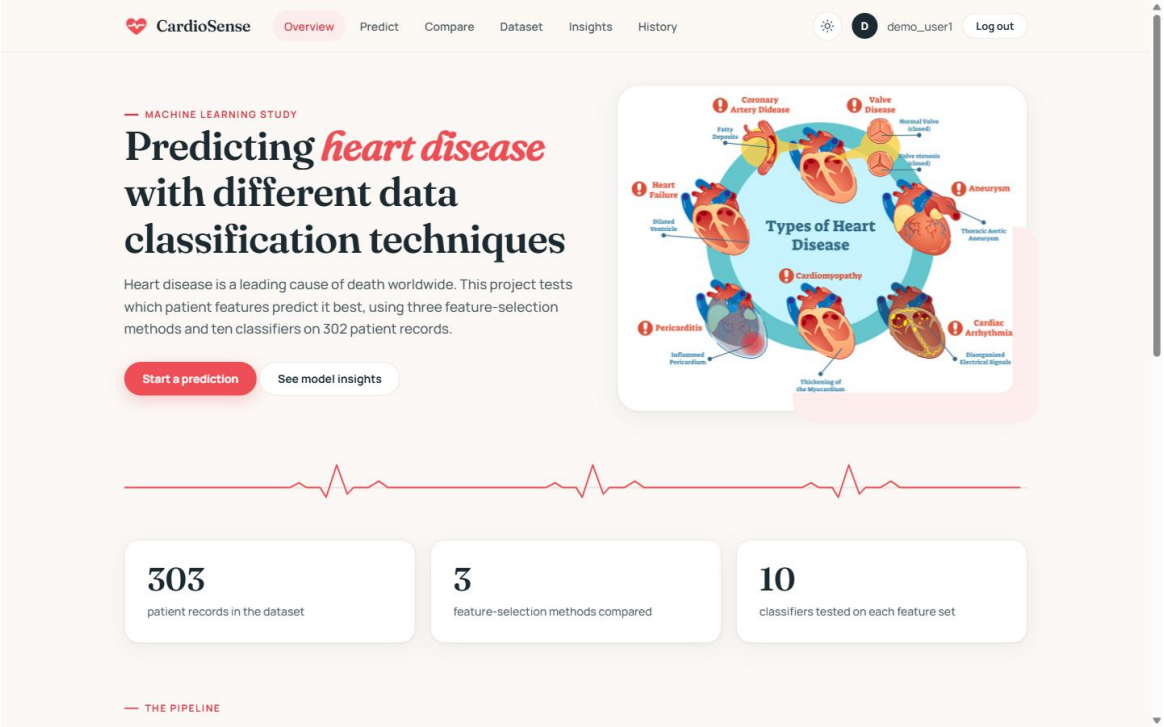


Figure 6.1: Overview page with the training pipeline

After logging in, the user opens **Predict**, chooses a method and enters the patient's values. Yes/no inputs are toggles, coded inputs are dropdowns with clinical names, and numbers show their unit and the range found in the dataset. Every input has an (i) note.

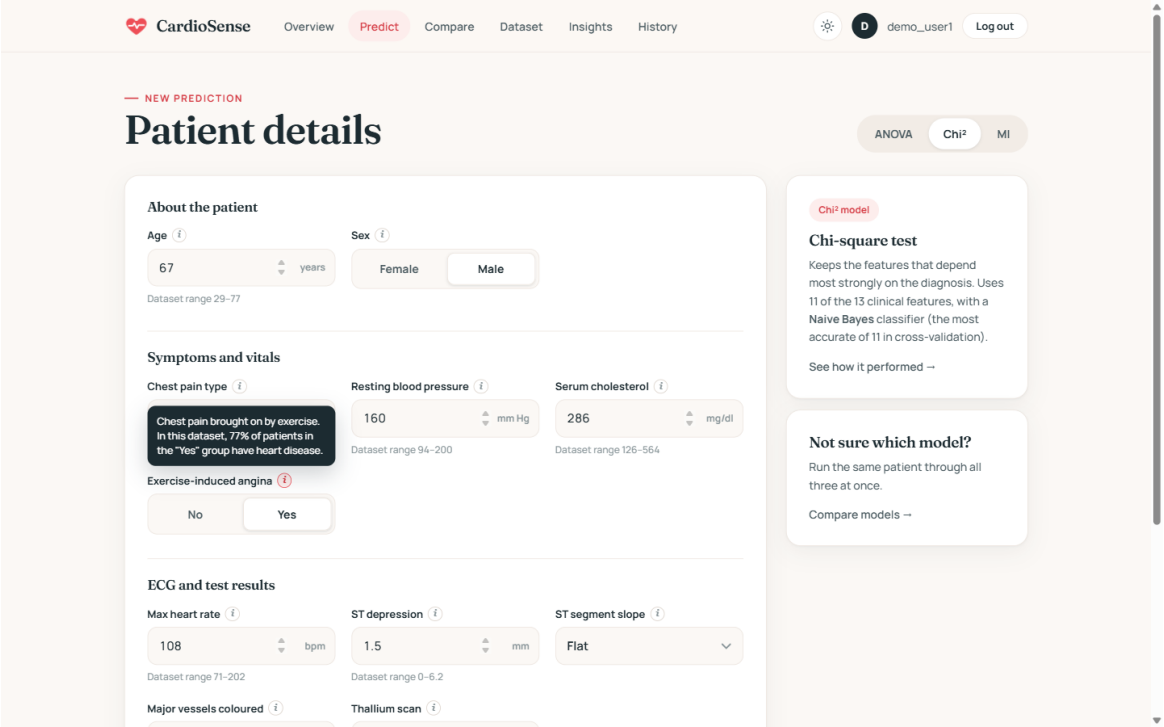


Figure 6.2: Prediction form with an info note open

The result shows the estimated risk, the verdict and, for a dataset patient, the real diagnosis and whether the model was right. Below it, each value is marked as pointing toward or away from heart disease.

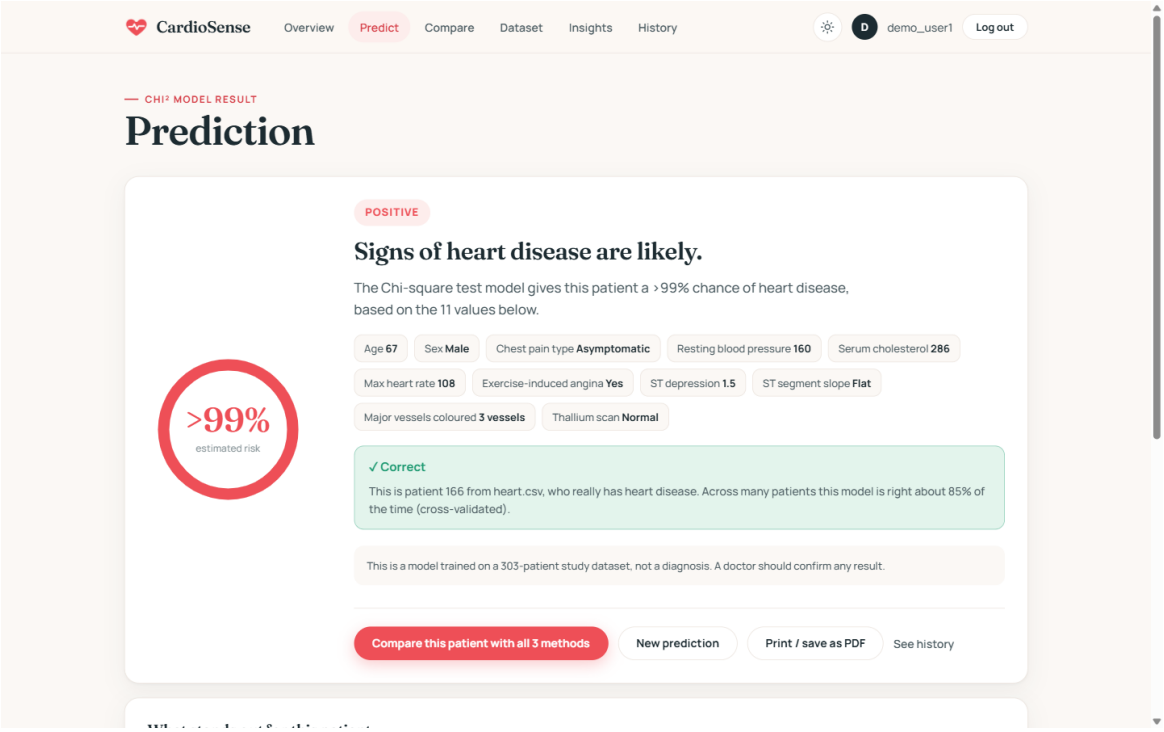


Figure 6.3: Prediction result for dataset patient 166

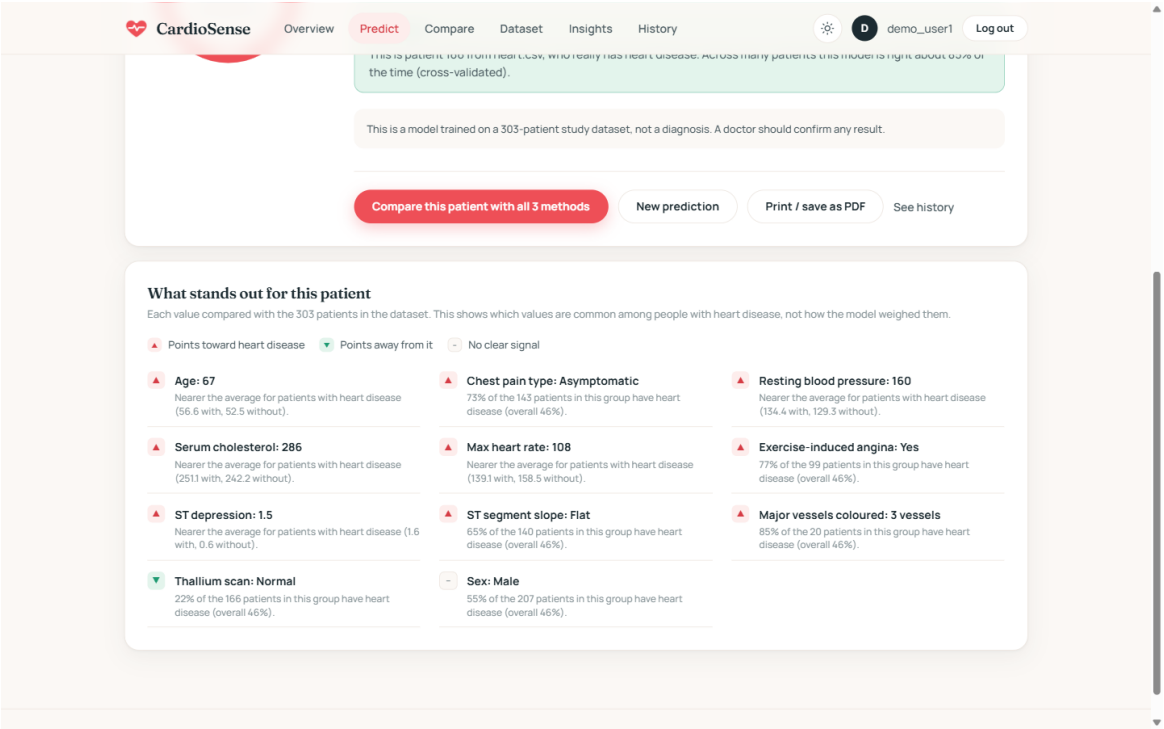


Figure 6.4: "What stands out for this patient"

Compare this patient with all 3 methods runs the same patient through every method and shows each one's verdict, correctness, accuracy and ignored features.

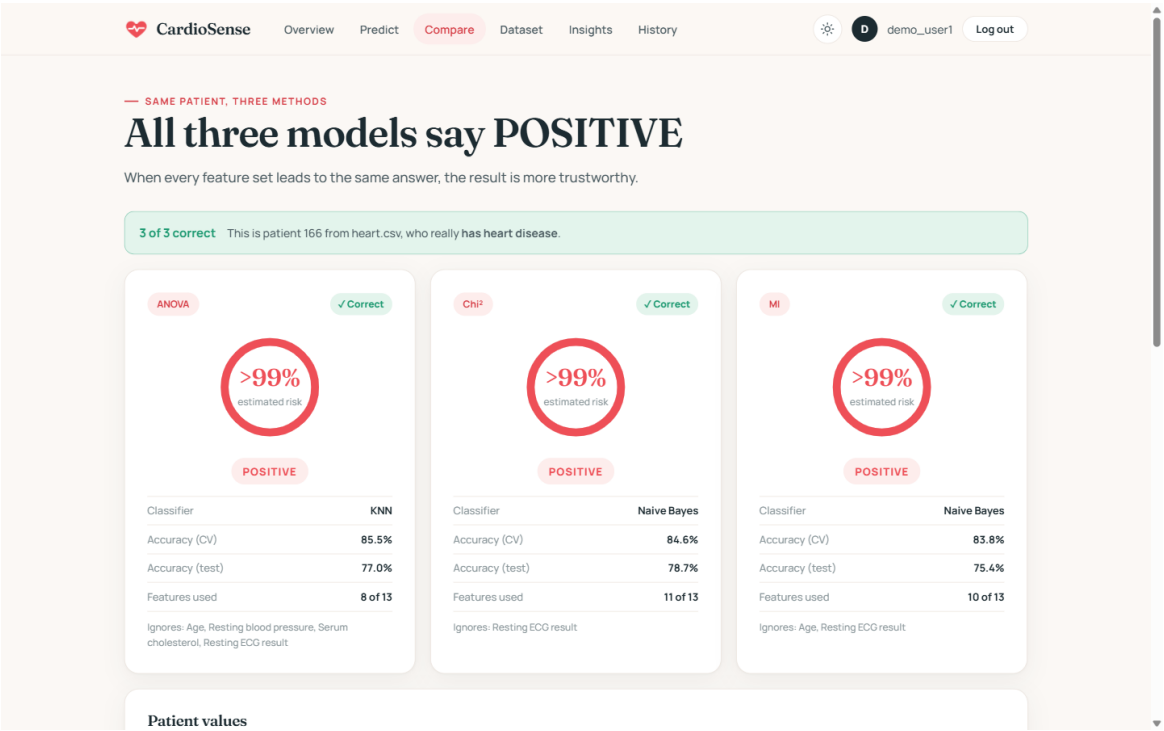


Figure 6.5: Comparison of the three methods on the same patient

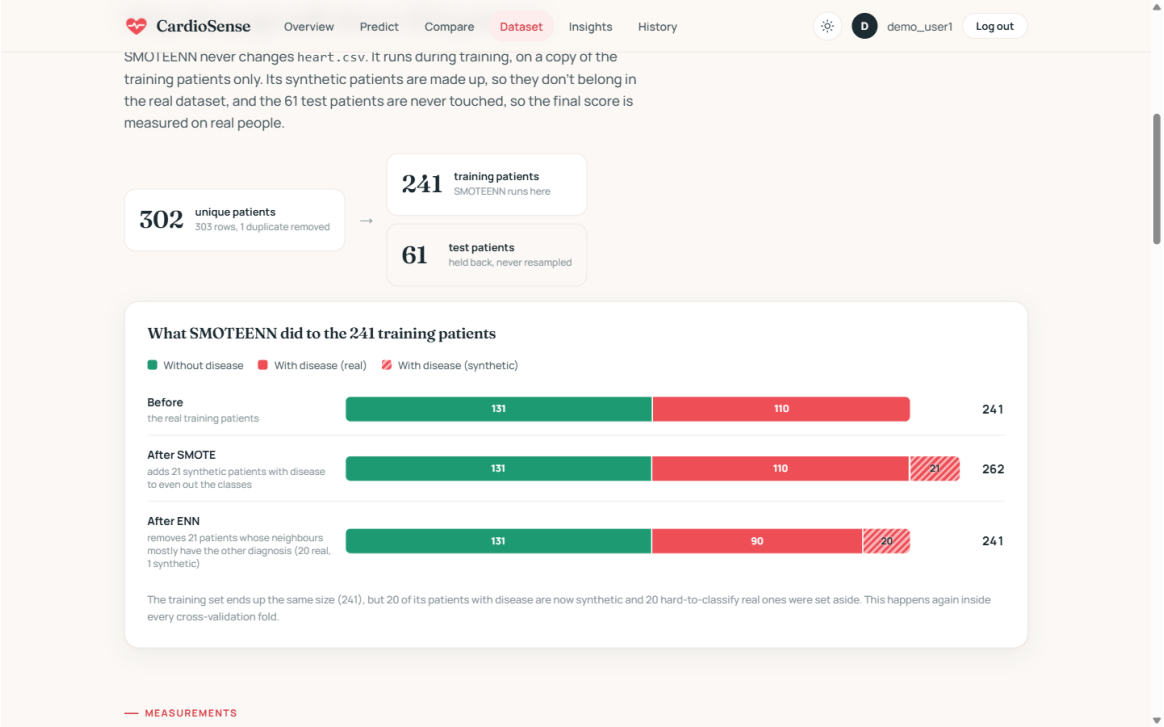


Figure 6.6: Dataset page: train/test split and SMOTEENN stages

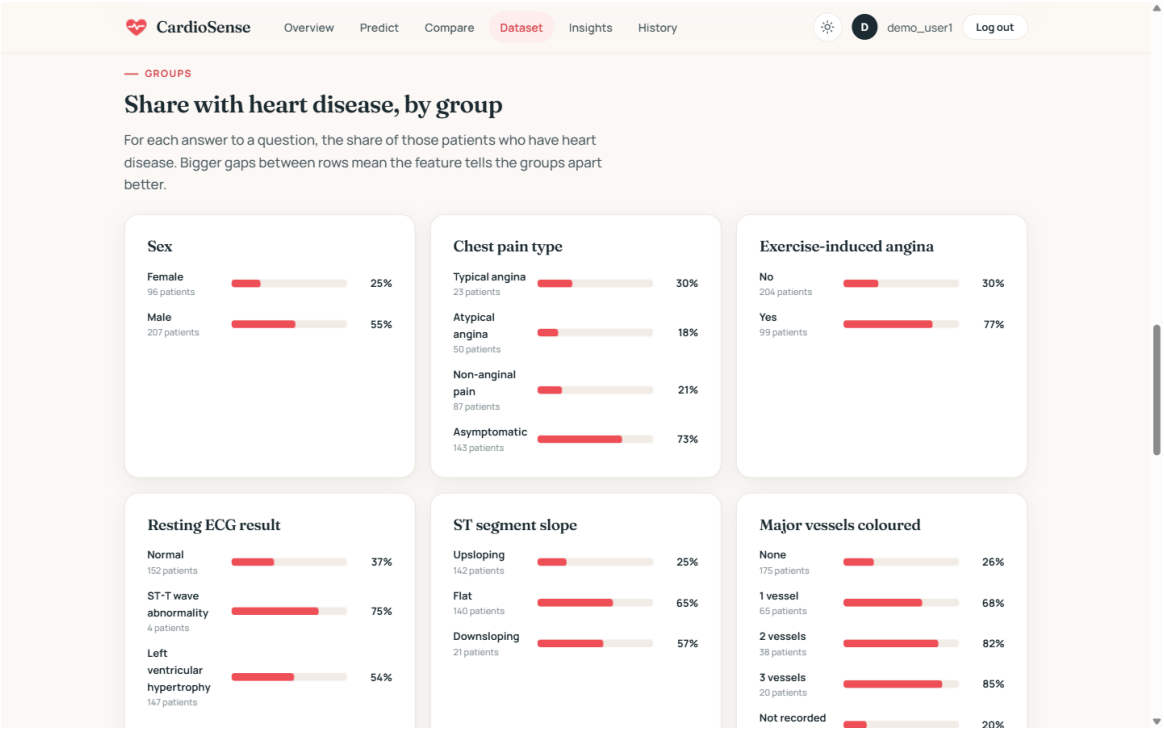


Figure 6.7: Dataset page: share of patients with heart disease by group

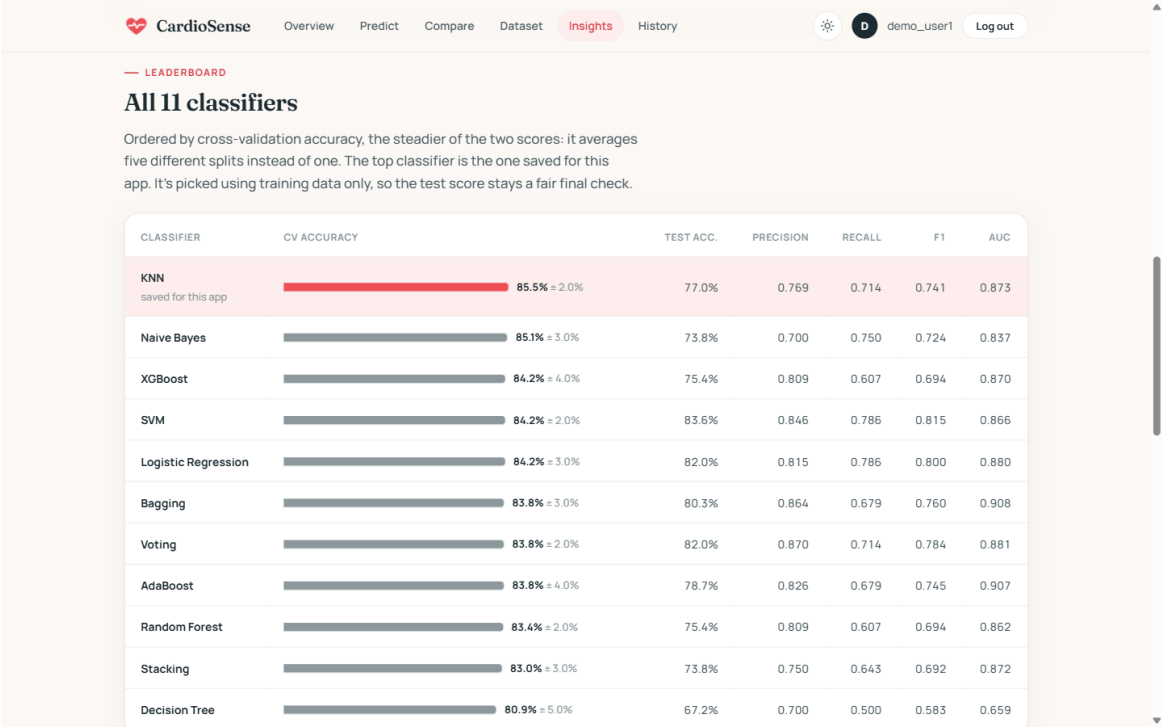


Figure 6.8: Insights: leaderboard of the eleven classifiers

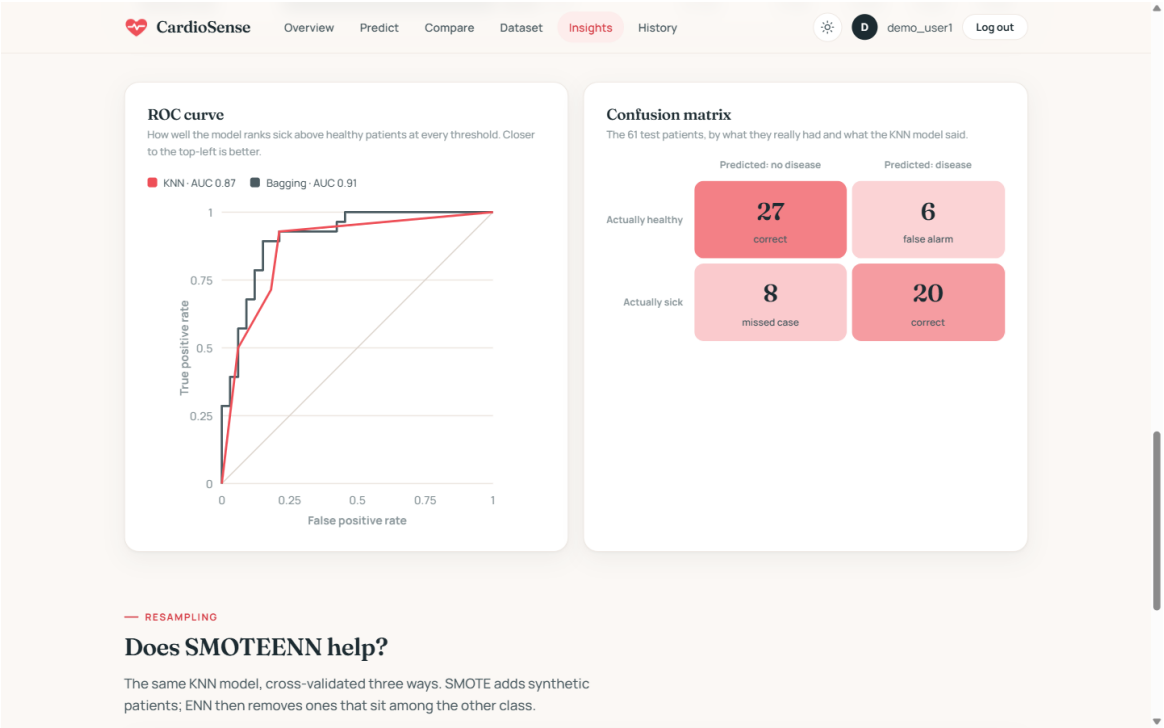


Figure 6.9: Insights: ROC curve and confusion matrix



Figure 6.10: Insights in dark mode: feature-selection scores

7. RESULT AND DISCUSSION

7.1 Classifier Results per Method

Tables 7.1 to 7.3 list all eleven classifiers for each method, ordered by cross-validated accuracy. Test-set figures are on the 61 held-out patients.

Table 7.1: ANOVA (8 features)

Classifier	CV acc. (± sd)	Test acc.	Precision	Recall	F1	AUC
KNN (saved)	85.5% ± 2.3	77.0%	0.769	0.714	0.741	0.873
Naive Bayes	85.1% ± 2.8	73.8%	0.700	0.750	0.724	0.837
XGBoost	84.3% ± 3.8	75.4%	0.809	0.607	0.694	0.870
SVM	84.2% ± 2.1	83.6%	0.846	0.786	0.815	0.866
Logistic Regression	84.2% ± 3.1	82.0%	0.815	0.786	0.800	0.880
Bagging	83.8% ± 3.0	80.3%	0.864	0.679	0.760	0.908
Voting	83.8% ± 2.4	82.0%	0.870	0.714	0.784	0.881
AdaBoost	83.8% ± 3.6	78.7%	0.826	0.679	0.745	0.907
Random Forest	83.4% ± 1.8	75.4%	0.809	0.607	0.694	0.862
Stacking	83.0% ± 3.1	73.8%	0.750	0.643	0.692	0.872
Decision Tree	80.9% ± 4.7	67.2%	0.700	0.500	0.583	0.659

Table 7.2: Chi-square (11 features)

Classifier	CV acc. (± sd)	Test acc.	Precision	Recall	F1	AUC
Naive Bayes (saved)	84.6% ± 2.1	78.7%	0.759	0.786	0.772	0.850
Logistic Regression	83.8% ± 3.1	83.6%	0.821	0.821	0.821	0.882
Voting	82.2% ± 4.1	82.0%	0.870	0.714	0.784	0.890
Bagging	82.1% ± 3.2	82.0%	0.870	0.714	0.784	0.897
KNN	81.7% ± 4.9	82.0%	0.815	0.786	0.800	0.853
SVM	81.3% ± 4.1	83.6%	0.846	0.786	0.815	0.870
AdaBoost	81.3% ± 3.0	78.7%	0.857	0.643	0.735	0.908
XGBoost	80.5% ± 3.9	78.7%	0.826	0.679	0.745	0.890
Random Forest	80.1% ± 3.3	80.3%	0.864	0.679	0.760	0.879
Stacking	79.7% ± 3.1	73.8%	0.833	0.536	0.652	0.889
Decision Tree	75.5% ± 3.1	70.5%	0.727	0.571	0.640	0.695

Table 7.3: Mutual Information (10 features)

Classifier	CV acc. (± sd)	Test acc.	Precision	Recall	F1	AUC
Naive Bayes (saved)	83.8% ± 2.4	75.4%	0.724	0.750	0.737	0.843
Bagging	83.0% ± 4.1	82.0%	0.870	0.714	0.784	0.905
AdaBoost	82.2% ± 3.1	77.0%	0.850	0.607	0.708	0.912
Logistic Regression	82.2% ± 3.9	83.6%	0.821	0.821	0.821	0.887
Stacking	81.8% ± 3.5	78.7%	0.857	0.643	0.735	0.884
XGBoost	81.7% ± 3.3	78.7%	0.826	0.679	0.745	0.897
SVM	81.3% ± 3.6	83.6%	0.846	0.786	0.815	0.875
Voting	80.9% ± 3.2	83.6%	0.875	0.750	0.808	0.882
Random Forest	80.5% ± 2.7	78.7%	0.826	0.679	0.745	0.869
Decision Tree	80.1% ± 6.5	68.8%	0.714	0.536	0.612	0.677
KNN	79.3% ± 5.2	82.0%	0.815	0.786	0.800	0.839

7.2 Comparison of the Three Methods

Table 7.4: Saved model per method

Method	Classifier	CV accuracy	Test acc.	Recall	AUC	Confusion (TN FP / FN TP)
ANOVA	KNN	85.5% ± 2.3	77.0%	71.4%	0.87	27 6 / 8 20
Chi-square	Naive Bayes	84.6% ± 2.1	78.7%	78.6%	0.85	26 7 / 6 22
Mutual Information	Naive Bayes	83.8% ± 2.4	75.4%	75.0%	0.84	25 8 / 7 21

ANOVA with KNN has the highest cross-validated accuracy, but the three saved models are only 1.6 percentage points apart, while one model's score moves by about ±2.3 points between folds. The difference is therefore **within normal variation**: no method is clearly better. Chi-square catches the most patients with disease on the test set (recall 78.6%), which matters most for screening, while ANOVA reaches the same accuracy with only 8 of the 13 measurements.

Some classifiers score higher than the saved model on the single test split; for example, SVM and logistic regression reach 83.6% test accuracy with ANOVA features against 77.0% for KNN. The saved model is deliberately chosen by cross-validation, not by test score: with only 61 test patients, one patient changes accuracy by 1.6 points, and choosing the model that happens to do best on the test set would turn the test set into part of training and make its score optimistic. Cross-validation over five splits is the steadier guide, and the test set remains an untouched final check.

7.3 Effect of SMOTEENN

Table 7.5: Cross-validated accuracy of the saved model with different resampling

Resampling	ANOVA	Chi-square	Mutual Information
No resampling	84.6%	83.0%	82.2%
SMOTEENN (default) — 65 rows left	72.6%	73.0%	76.3%
SMOTEENN (tuned, used)	85.5%	84.6%	83.8%

The tuned SMOTEENN gives a small, consistent gain over no resampling for all three methods. The default SMOTEENN is clearly harmful here because it discards most of the training data. Since the dataset is already fairly balanced (164 healthy vs 138 with disease), large gains were not expected.

7.4 Stacking Experiment

Table 7.6: Additional stacking ensembles (cross-validated accuracy)

Stacking ensemble (meta-model: logistic regression)	ANOVA	Chi-square	MI
Saved models (best of 11)	**85.5%**	**84.6%**	**83.8%**
Naive Bayes + KNN + Logistic Regression	83.8%	82.6%	83.0%
Naive Bayes + KNN + LR + SVM + Random Forest	83.4%	81.3%	82.6%
Naive Bayes + LR + Random Forest + Extra Trees	83.4%	83.0%	83.4%

To test whether a stronger ensemble could lift any method, three further stacking ensembles were cross-validated in the same way on all three methods (`experiment_stacking.py`). None beat the saved models. On a dataset of this size, simple well-chosen models are as good as complex ensembles.

7.5 Comparison with the Original Notebooks

The original notebooks reported 100% accuracy for the stacking model. That figure came from applying SMOTE to the whole dataset before splitting, so near-copies of test patients were in the training data, and AUC was computed on training data. With the test set separated first, the same stacking model scores 73.8–78.7% on the test set; picking the best classifier by cross-validation instead raises recall from 54–64% to 71–79%. Honest figures of about 84–86% are lower but reproducible.

7.6 Cost Analysis

Table 7.7: Cost analysis

Item	Cost
Software (Python, Flask, scikit-learn, imbalanced-learn, LightGBM, XGBoost, SQLite)	₹0 (open source)
Dataset (UCI Cleveland heart disease)	₹0 (public, CC BY 4.0)
Development hardware	₹0 (existing laptop)
Email OTP (Gmail SMTP)	₹0 within normal sending limits
Optional public deployment (small cloud server and domain)	[About ₹500–1,000 per month; check current provider prices]
Total for development and demonstration	**₹0**

8. CONCLUSION

This project built CardioSense, a heart-disease prediction system that compares ANOVA, Chi-square and Mutual Information feature selection across eleven classifiers. Its main results are: a data audit that found and corrected reversed labels in a widely used copy of the dataset; a leakage-free pipeline in which the test set is separated before any learning step; and a fair comparison showing cross-validated accuracies of 85.5%, 84.6% and 83.8%, within normal variation of each other. A small set of eight routine measurements selected by ANOVA performs as well as larger sets.

The web application turns these models into an understandable tool: it shows risk, compares methods on the same patient, checks predictions against the real diagnosis for dataset patients, and explains each value against the dataset, while handling accounts securely.

Future work. (1) Calibrate the predicted probabilities so that risk percentages are reliable (e.g. with CalibratedClassifierCV). (2) Validate the models on data from other hospitals, handling missing values. (3) Deploy behind HTTPS with rate limiting of OTP emails and CSRF protection. (4) Explore model-specific explanations such as SHAP values.

9. REFERENCES

[1] World Health Organization, "Cardiovascular diseases (CVDs) — Fact sheet," 2021. [Online]. Available: https://www.who.int/news-room/fact-sheets/detail/cardiovascular-diseases-(cvds)

[2] R. Detrano et al., "International application of a new probability algorithm for the diagnosis of coronary artery disease," American Journal of Cardiology, vol. 64, no. 5, pp. 304–310, 1989.

[3] A. Janosi, W. Steinbrunn, M. Pfisterer and R. Detrano, "Heart Disease," UCI Machine Learning Repository, 1988. doi: 10.24432/C52P4X.

[4] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," ACM Transactions on Knowledge Discovery from Data, vol. 6, no. 4, 2012.

[5] I. Guyon and A. Elisseeff, "An introduction to variable and feature selection," Journal of Machine Learning Research, vol. 3, pp. 1157–1182, 2003.

[6] A. Kraskov, H. Stögbauer and P. Grassberger, "Estimating mutual information," Physical Review E, vol. 69, 066138, 2004.

[7] Z. Noroozi, A. Orooji and L. Erfannia, "Analyzing the impact of feature selection methods on machine learning algorithms for heart disease prediction," Scientific Reports, vol. 13, 22588, 2023.

[8] N. V. Chawla, K. W. Bowyer, L. O. Hall and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," Journal of Artificial Intelligence Research, vol. 16, pp. 321–357, 2002.

[9] D. L. Wilson, "Asymptotic properties of nearest neighbor rules using edited data," IEEE Transactions on Systems, Man, and Cybernetics, vol. 2, no. 3, pp. 408–421, 1972.

[10] G. E. Batista, R. C. Prati and M. C. Monard, "A study of the behavior of several methods for balancing machine learning training data," ACM SIGKDD Explorations, vol. 6, no. 1, pp. 20–29, 2004.

[11] G. Lemaître, F. Nogueira and C. K. Aridas, “Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning,” Journal of Machine Learning Research, vol. 18, no. 17, pp. 1–5, 2017.

[12] L. Breiman, “Random forests,” Machine Learning, vol. 45, pp. 5–32, 2001.

[13] Y. Freund and R. E. Schapire, “A decision-theoretic generalization of on-line learning and an application to boosting,” Journal of Computer and System Sciences, vol. 55, no. 1, pp. 119–139, 1997.

[14] T. Chen and C. Guestrin, “XGBoost: A scalable tree boosting system,” in Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining, pp. 785–794, 2016.

[15] G. Ke et al., “LightGBM: A highly efficient gradient boosting decision tree,” in Advances in Neural Information Processing Systems 30, 2017.

[16] D. H. Wolpert, “Stacked generalization,” Neural Networks, vol. 5, no. 2, pp. 241–259, 1992.

[17] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

APPENDIX A – SAMPLE CODE

Selected excerpts from the project source. The full code is in the project folder.

A.1 Tuned SMOTEENN (train.py)

```
def _smoteenn(X, y, counts=None):
    """SMOTEENN tuned for this data, done as its two steps so we can report what each one
        did.
    Features are scaled first so cholesterol (hundreds) doesn't drown out yes/no features in
        the
    distance checks; SMOTENC keeps categories as real codes instead of inventing values
        like
    cp = 1.37; ENN 'mode' only removes a patient when most of its neighbours disagree.
    The default SMOTEENN threw away most of the training set.
    Pass a dict as `counts` to get the class sizes before, after SMOTE and after ENN."""
    scaler = StandardScaler().fit(X)
    X_s, y_s = SMOTENC(categorical_features=CATEGORICAL,
        random_state=SEED).fit_resample(scaler.transform(X), y)
    enn = EditedNearestNeighbours(kind_sel='mode')
    X_e, y_e = enn.fit_resample(X_s, y_s)
    if counts is not None:
        counts.update({stage: np.bincount(labels, minlength=2).tolist()
                       for stage, labels in (('before', y), ('after_smote', y_s),
                           ('after_enn', y_e))})
        kept = set(enn.sample_indices_.tolist())  # SMOTENC appends synthetic rows after the
            len(y) real ones
        counts.update(synthetic_added=len(y_s) - len(y),
                      synthetic_kept=sum(i >= len(y) for i in kept),
                      real_removed=sum(i not in kept for i in range(len(y))))
    X_res = scaler.inverse_transform(X_e)
    X_res[:, CATEGORICAL] = np.round(X_res[:, CATEGORICAL])
    return np.clip(X_res, 0, None), y_e  # clip guards chi2, which needs non-negative inputs
```

A.2 Cross-validated training and model choice (train.py)

```
        # 3. train and score all eleven classifiers
        results, roc, fitted = [], {}, {}
        for name, clf in classifiers().items():
            clf.fit(Xs_res, y_res)
            fitted[name] = clf
            pred, prob = clf.predict(Xs_te), clf.predict_proba(Xs_te)[:, 1]
            # cross-validation repeats resampling + selection inside every fold, so no fold
                sees its own test rows
            cv_pipe = Pipeline([('resample', resampler()), ('select', SelectKBest(score_fn,
                k=k)),
                                ('model', classifiers()[name])])
            cv_acc = cross_val_score(cv_pipe, X_tr, y_tr, cv=cv, scoring='accuracy')
            results.append(dict(name=name, accuracy=accuracy_score(y_te, pred),
                precision=precision_score(y_te, pred),
                                recall=recall_score(y_te, pred), f1=f1_score(y_te, pred),
                                    auc=roc_auc_score(y_te, prob),
                                cv_mean=cv_acc.mean(), cv_std=cv_acc.std()))
            fpr, tpr, _ = roc_curve(y_te, prob)
            roc[name] = [[round(a, 4), round(b, 4)] for a, b in zip(fpr, tpr)]

        # 4. keep the best by cross-validated accuracy (ties: steadier, i.e. smaller spread
            across folds)
        best = max(results, key=lambda r: (round(r['cv_mean'], 3), -r['cv_std']))
        joblib.dump(fitted[best['name']], path)
        matrix = confusion_matrix(y_te, fitted[best['name']].predict(Xs_te)).tolist()
```

A.3 Input validation (app.py)

```
def read_inputs(fields, form):
    """Return the form values as floats in model order, or raise ValueError with a readable
```

```
        message."""
    values = []
    for f in fields:
        label, _, rule = FIELDS[f]
        try:
            v = float(form[f])
        except (KeyError, ValueError):
            raise ValueError(f'{label} needs a number.')
        ok = (rule[0] <= v <= rule[1]) if isinstance(rule, tuple) else v in CHOICE_NAMES[f]
        if not ok:
            raise ValueError(f'{label} is out of range.')
        values.append(v)
    return values
```

A.4 Prediction route (app.py)

```
@app.route('/predict/<model>', methods=['GET', 'POST'])
@login_required
def predict(model):
    m = get_model(model)
    if request.method == 'GET':
        return render_template('predict.html', key=model, m=m,
            rows=dataset_rows(m['fields']))
    try:
        values = read_inputs(m['fields'], request.form)
    except ValueError as e:
        flash(str(e))
        return redirect(url_for('predict', model=model))
    positive, risk = run_model(model, values)
    save_prediction(model, m['fields'], values, positive, risk)
    row = matching_row(request.form, dict(zip(m['fields'], values)))
    return render_template('result.html', key=model, m=m, positive=positive, risk=risk,
                           inputs=list(zip(m['fields'], values)), row=row,
                           actual=None if row is None else DATASET[row]['target'] == 1)
```